

RAPPORT DE STAGE M2

INFORMATIQUE

Développement d'une application d'incrustation visuelle
d'équipements thermiques et de climatisations par
Intelligence Artificielle



Soitel Rémi
INFO1002, M2 INFO

Table des matières

I.	Remerciement	2
II.	Introduction	2
III.	Objectif du Projet	3
A.	Contexte et Problématique	3
B.	Les Différents choix d'Implémentations.....	3
1.	Utilisation d'un Autocollant Physique	3
2.	Le choix d'OpenCV	4
3.	Le Choix de Flutter	5
IV.	Implémentation	5
A.	Implémentation des Intelligences Artificielles.....	5
1.	IA pour la Détection de l'Autocollant.....	6
2.	Inpainting pour Effacer l'Autocollant du Mur	7
B.	Traitement OpenCV	8
1.	Projection Perspective et Incrustation de Base	8
2.	Éclairage, Ombres Portées et Colorimétrie Adaptative	9
3.	Modélisation du Volume/Profondeur	9
C.	Création de l'Application et de l'Interface Utilisateur.....	10
1.	Architecture et Gestion d'État	10
2.	Outil de Capture Gyroscopique	10
3.	Incrustation des Goulottes	11
4.	Interface Utilisateur	11
V.	Perspective d'Amélioration	12
VI.	Conclusion.....	12

I. Remerciement

Je tiens tout d'abord avant de commencer exprimer mes remerciements à toutes les personnes qui ont contribué au bon déroulement de ce stage.

Tout d'abord, je remercie monsieur le Président Stéphane SOITEL qui fut mon tuteur de stage pour son encadrement, ses remarques et son soutien le long du stage.

Je souhaite également remercier monsieur Hacène Fouchal, Responsable de formation du Master Informatique qui nous a offert la possibilité de faire ce stage afin d'y appliquer nos connaissances.

Enfin, je remercie aussi toute l'équipe de l'entreprise Chauffage Der Energies pour leur accueil. Leur esprit d'équipe a grandement facilité mon intégration et m'ont permis de travailler dans un environnement adapté à l'apprentissage et au bon déroulement du projet.

II. Introduction

Ce rapport présente les travaux réalisés au cours de mon stage de fin d'étude pour le Master 2 Informatique parcours Intelligence Artificielle effectué au sein de l'Entreprise Chauffage Der Energies. Ce stage constitue l'aboutissement de mon cursus académique et s'inscrit dans une volonté d'appliquer concrètement les méthodes d'apprentissage automatique et de traitement d'image à des besoins terrains.

L'entreprise **Chauffage Der Energies** est spécialisée dans la proposition et l'installation de solutions thermiques et climatiques (pompes à chaleur, climatisations, unités extérieures, etc...). Dans ce secteur d'activité, les commerciaux et les artisans rencontrent un défi lors de la phase de devis : la difficulté pour le client final de se projeter. En effet, il est souvent complexe pour un particulier d'imaginer l'impact visuel, l'encombrement et le rendu esthétique d'un équipement une fois installé sur ses propres murs ou dans son jardin.

Pour répondre à cette problématique, l'objectif principal de ce stage a été de concevoir et de développer de zéro une application mobile innovante d'aide à la vente. Ce simulateur permettrait à partir d'une simple photographie prise chez le client d'y incruster virtuellement les équipements du catalogue de l'entreprise avec un haut niveau de réalisme grâce à un autocollant. Le cœur de l'application devait se reposer sur la synergie entre des modèles d'Intelligence Artificielle embarqués (réseaux de neurones pour la détection d'environnement et la reconstruction d'image) et d'algorithmes de traitement d'image. Ces derniers auront permis d'automatiser des tâches complexes telles que la gestion dynamique des perspectives, la génération de volumes proportionnés, le calcul d'ombres portées adaptatives ou encore la conception d'un routage de goulottes d'installation.

À travers les différentes parties de ce rapport, nous détaillerons les étapes de conception, de recherche et de développement de cette application. Dans un premier temps nous présenterons plus en détail le contexte du projet et le cahier des charges. Nous justifierons ensuite nos choix technologiques pour répondre aux contraintes du développement mobile et de l'IA embarquée. Nous aborderons par la suite le cœur de notre travail, structuré autour de l'implémentation des modèles d'Intelligence Artificielle, du traitement d'image mathématique et de la création d'une interface utilisateur ergonomique. Enfin, nous proposerons des perspectives d'amélioration pour l'outil avant de conclure sur les apports professionnels de cette expérience.

III. Objectif du Projet

A. Contexte et Problématique

L'entreprise Chauffage Der Energies opère sur un marché concurrentiel où la vente de solutions thermiques ne repose plus uniquement sur la performance technique des équipements mais également sur la qualité du conseil et la capacité à rassurer le client final. L'installation d'une pompe à chaleur ou d'une climatisation représente un investissement important pour un foyer et peut être perçu comme une transformation structurelle du logement.

Dans ce contexte, le processus de vente traditionnelle, qui s'appuie souvent sur des catalogues papier ou des photos génériques de constructeurs atteint ses limites. Les clients éprouvent certaine difficulté à se projeter, il leur est complexe d'appréhender le volume réel d'un appareil, son intégration architecturale ou encore l'encombrement des tuyaux sur une façade ou un mur intérieur. Ce manque de visibilité peut constituer un frein à la décision d'achat, le client craignant une « dégradation esthétique » de son lieu de vie. Pour l'entreprise, l'enjeu était donc de mettre en place un levier de conversion technologique capable de lever ces incertitudes en offrant une visualisation personnalisée, immédiate et fiable de n'importe quel équipement possible d'installations par l'entreprise.

L'adoption d'une solution numérique pour répondre à ce besoin commercial impose des contraintes techniques majeures. Tout d'abord, le caractère itinérant de l'activité des installateurs nécessite une application autonome, elle doit être capable de fonctionner sur le terrain souvent dans des zones où la connexion internet est instable. Par conséquent, tous les processus de traitement doivent être intégralement embarqués dans le terminal mobile sans dépendre de serveurs distants. Ensuite l'application doit impérativement respecter les règles de perspective, gérer les occultations lumineuses et intégrer les contraintes techniques de pose (notamment la gestion des goulottes et les distances de sécurité des machines). Le défi réside ici dans la capacité à créer un rendu "photoréaliste" où l'équipement virtuel se fond naturellement dans l'image réelle, comme s'il avait été photographié directement sur place et en étant à la bonne taille en permanence. Enfin, s'ajoute à ça que la fluidité de l'utilisation est déterminante. Un artisan ou un commercial ne peut se permettre de consacrer plusieurs minutes à paramétrer un logiciel complexe chez un client. Le processus, de la prise de vue à l'affichage du rendu final doit être le plus fluide possible avec une interface utilisateur simple. Cela implique aussi une optimisation rigoureuse des algorithmes pour garantir une expérience utilisateur fluide sans latence malgré la lourdeur des traitements mathématiques nécessaires pour la manipulation des images haute résolution en temps réel.

Nous avons donc réfléchi avec l'équipe pour savoir comment implémenter le tout et ainsi fait plusieurs choix d'implémentation.

B. Les Différents choix d'Implémentations

Pour concevoir une solution capable de répondre aux exigences, plusieurs orientations technologiques ont dû être validées. La robustesse du simulateur repose sur trois piliers fondamentaux que nous avons choisis.

1. Utilisation d'un Autocollant Physique

La première problématique rencontrée lors de la phase de conception fut celle de la précision géométrique et de l'échelle. Pour qu'une incrustation soit réaliste il ne suffit pas de poser une image

sur une autre, il faut que l'équipement respecte la perspective du mur photographié et que ses dimensions soient physiquement cohérentes. Nous avons donc opté pour l'utilisation d'un référentiel physique qui servira de marqueur.

Ce choix, bien que nécessitant une étape préalable de placement par l'artisan, garantit une fiabilité exemplaire. L'autocollant possède des dimensions connues ce qui fait qu'il devient un point d'ancrage absolu dans l'espace. Il permet à l'algorithme d'IA de calculer instantanément la bonne taille de l'équipement (Qui sont connu) en fonction de la taille de l'autocollant en faisant une nouvelle échelle avec les pixels. Cette méthode offre une stabilité de rendu que les solutions basées sur la simple détection de plans ne pourraient égaler dans des conditions d'éclairage variables ou sur des surfaces peu texturées.

Pour assurer la reconnaissance de cet autocollant en temps réel, nous avons fait le choix d'utiliser un modèle de détection d'objet de la famille YOLO et plus spécifiquement une architecture de type « Small » (ou Nano au début du projet). La contrainte principale de notre cahier des charges étant l'exécution en local sur un terminal mobile dépourvu de connexion internet, un modèle d'intelligence artificielle trop profond aurait provoqué des temps d'inférence inacceptables et saturé la RAM de l'appareil. Le choix d'une architecture allégée offre un très bon compromis, l'IA conserve une excellente précision pour détecter une forme géométrique simple et normée comme notre autocollant tout en garantissant un traitement quasi instantané et une faible consommation énergétique pour l'utilisateur sur le terrain.

Le choix des caractéristiques physiques de ce marqueur ne s'est pas fait au hasard. Nous avons opté pour un autocollant d'une dimension exacte de 10 cm par 5 cm, arborant une couleur vert fluo plutôt prononcée. Cette taille offre un compromis idéal car elle est suffisamment discrète pour ne pas polluer visuellement la prise de vue initiale par l'artisan, tout en restant parfaitement nette pour les capteurs des smartphones. La couleur vert fluo garantit un contraste colorimétrique maximal avec les teintes classiques des murs intérieurs et extérieurs (blanc, beige, brique, crépi). Ce fort contraste est un atout géométrique et visuel majeur pour faciliter la délimitation de la boîte englobante lors de l'entraînement et de l'inférence du modèle de détection.



Rouleau d'autocollant utilisé

2. Le choix d'OpenCV

La manipulation des flux d'images haute résolution et la génération de volumes exigeaient une bibliothèque de traitement d'image mature, performante et capable d'effectuer des opérations matricielles intensives. OpenCV s'est imposé comme le choix naturel. Sa capacité à gérer nativement des opérations de type transformation perspective, convolution, calcul de contours et blending alpha a été déterminante.

OpenCV agit ici comme le moteur de rendu de l'application, c'est-à-dire qu'après que l'IA a identifié la position de l'autocollant, OpenCV prend le relais pour déformer le modèle de l'équipement. Cette bibliothèque offre le compromis idéal entre performance brute et fonctionnalité permettant de réaliser des opérations de composition d'images complexes telles que l'ombre portée adaptative et le

flou de dégradation photographique qui sont nécessaires pour effacer la frontière entre le réel et le virtuel.

3. Le Choix de Flutter

Le développement d'une application professionnelle destinée au terrain imposait deux contraintes majeures : une interface utilisateur réactive capable de gérer des interactions fluides et une architecture capable de traiter des calculs lourds sans interruption du flux visuel. Le Framework Flutter basé sur le langage Dart a été sélectionné pour sa capacité à produire des interfaces hautement personnalisables et performantes. Et bien que nous n'ayons jamais utilisé Flutter auparavant (On utilisait surtout Android Studio), le langage Dart est très similaire à du C++ que nous connaissons bien et un des avantages de flutter c'est que nous n'avons pas à gérer l'application en fonction de l'OS du téléphone, il gère tout tout seul.

Ainsi avec ses choix établis au début du projet, nous avons pu commencer l'implémentation de celui-ci.

IV. Implémentation

La phase d'implémentation du projet a nécessité une approche méthodique, dictée par la complexité de combiner des algorithmes d'apprentissage automatique avec des traitements mathématiques lourds sur un appareil mobile. Avant de nous lancer dans le développement de l'architecture finale et de l'intelligence artificielle, le projet a débuté par la réalisation d'une Proof of Concept utilisant exclusivement la bibliothèque OpenCV sous environnement Python. Cette première étape expérimentale nous a permis de valider rapidement la faisabilité mathématique de la déformation de perspective et de l'incrustation basique d'une image sur une autre.

Cependant, la migration de ce code Python fonctionnel vers l'application mobile en langage Dart a représenté l'un des défis majeurs du projet. Le portage des algorithmes de traitement d'image a nécessité de lourdes adaptations, le wrapper *opencv_dart* pour Flutter présentant des limitations et des différences d'implémentation par rapport à la version native Python d'OpenCV. Ce n'est qu'une fois ces barrières techniques franchies et la validation géométrique acquise en Dart que nous avons pu entamer le développement de l'Intelligence Artificielle avant d'assembler l'application finale.

Une fois cette validation géométrique acquise, nous avons pu entamer le développement de l'Intelligence Artificielle avant ensuite d'entamer l'application finale. Cette construction s'est faite de manière chronologique : nous avons d'abord mis en place l'Intelligence Artificielle pour la reconnaissance et la préparation de l'environnement, puis nous sommes retournés sur OpenCV pour développer le véritable moteur de rendu 3D complexe, avant de consolider l'ensemble au sein de l'interface utilisateur dynamique en Flutter. Etant donné que j'étais seul à travailler sur le projet, il m'arriver de « slalomer » entre les parties malgré que nous allions décrire leurs implémentation de manière individuelle.

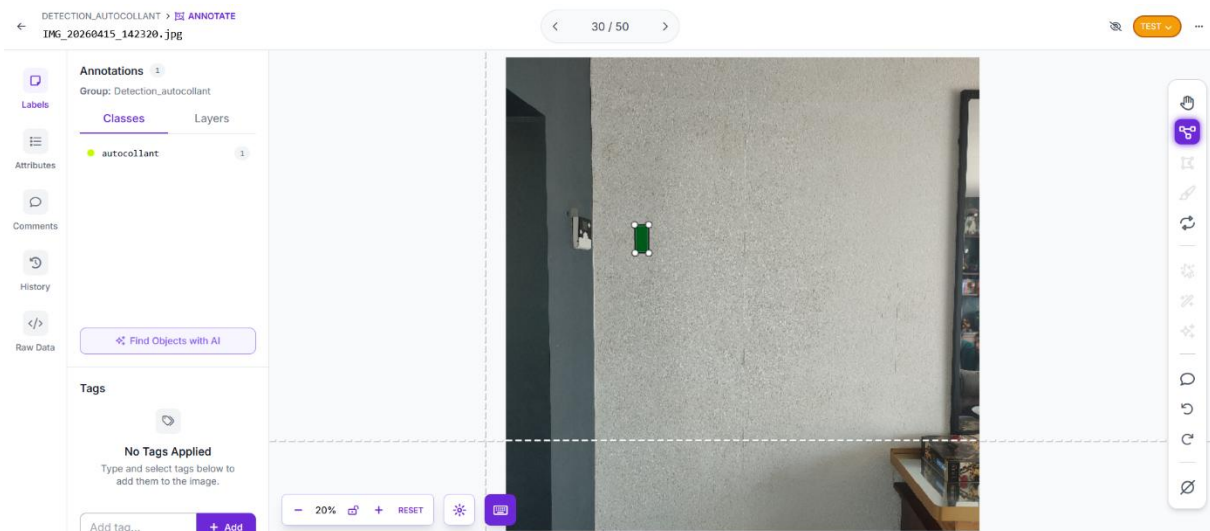
A. Implémentation des Intelligences Artificielles

Le rôle de l'Intelligence Artificielle dans notre application est d'agir comme les « yeux » du système : elle doit d'une part détecter précisément l'autocollant de repère placé sur le mur pour en extraire les coordonnées spatiales, et d'autre part effacer virtuellement cet autocollant de la photographie pour restituer un mur vierge et propre au client. Pour accomplir ces tâches, nous avons implémenté deux modèles distincts fonctionnant via TensorFlow Lite, chargés dès le lancement de

l'application grâce à un service dédié afin de les maintenir dans la mémoire vive du téléphone de l'artisan.

1. IA pour la Détection de l'Autocollant

La première étape de cette implémentation a consisté à intégrer un modèle de détection d'objets YOLOv8 entraîné spécifiquement pour détecter notre autocollant. Cependant, pour que ce modèle soit performant et adapté à notre environnement technique spécifique, nous n'avons pas pu nous reposer sur des modèles pré-entraînés standards, il a fallu constituer notre propre jeu de données de toutes pièces. Nous avons capturé une cinquantaine de photographies de l'autocollant dans diverses conditions d'éclairage, de contraste et sur différentes textures de murs. Ensuite, nous avons eu recours à la plateforme Roboflow pour procéder à l'annotation rigoureuse de ces images, c'est-à-dire la délimitation manuelle et précise de la zone cible ainsi que pour générer des variations d'images



Interface de Roboflow pour annoter les images du dataset

avec de la data augmentation. A noter qu'avec cette méthode on retourne les valeurs de la boîte englobante qui a détecté l'autocollant et les 4 coins précis qui délimitent l'autocollant, c'est pour pouvoir détecter les photos un peu pencher même si toutes les photos restent prises de face.

Ce travail préparatoire a considérablement renforcé la robustesse du modèle final avant son exportation vers TensorFlow Lite.

Il est important de souligner que toute cette phase de conception, d'entraînement et d'évaluation du modèle d'Intelligence Artificielle a été intégralement codée et exécutée en Python. Ce langage étant un peu considéré comme le standard incontesté en science des données, il nous a permis d'exploiter efficacement les bibliothèques d'apprentissage automatique de l'écosystème YOLO. Ce n'est qu'une fois le réseau de neurones testé et validé sous Python qu'il a été exporté et converti au format TensorFlow Lite (.tflite), un des seuls formats optimisés pour une exécution embarquée et hors-ligne au sein de notre application Flutter.

Une fois le modèle entraîné, la principale difficulté technique rencontrée côté code a résidé dans la préparation et la manipulation des données brutes de l'appareil photo du smartphone. Les photographies capturées étant d'une résolution très élevée, il a fallu concevoir un algorithme de prétraitement robuste pour redimensionner l'image au format strict requis par YOLO (une matrice de 1024x1024 pixels) et normaliser les valeurs colorimétriques. De plus, pour s'assurer que notre

application puisse s'adapter aux différentes architectures matérielles des téléphones (iOS, Android), il a fallu coder une logique capable de gérer dynamiquement l'ordre des canaux de la matrice (formats NHWC ou NCHW).

Le post-traitement des résultats a également présenté son lot de défis. Il a fallu trier rigoureusement les quatre points de détection fournis par l'IA à l'aide de formules mathématiques pour garantir que l'ordre (haut-gauche, haut-droit, bas-droit, bas-gauche) soit toujours exact. Sans ce tri géométrique, les calculs de perspective ultérieurs auraient généré des équipements inversés ou tordus (Avec des zones de détection qui s'auto-chevauchent). Pour garantir une fiabilité à toute épreuve, nous avons également programmé une solution de repli dans lequel si la confiance de détection de l'IA est insuffisante (Ici défini à moins de 92%), on utilise les valeurs de la boîte englobante au lieu des 4 coins.

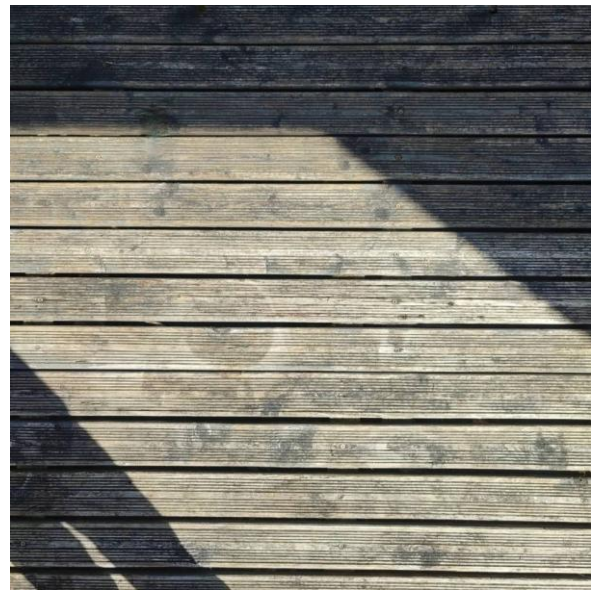
2. Inpainting pour Effacer l'Autocollant du Mur

Le choix d'utiliser un modèle d'IA générative pour cette tâche d'effacement découle directement d'une contrainte technique liée à notre migration vers Dart. Lors de notre PoC initial en Python, l'effacement du marqueur était réalisé de manière mathématique grâce à la fonction native *cv.inpaint()* d'OpenCV qui offrait des résultats satisfaisants par interpolation algorithmique. Or, lors du passage au développement mobile, nous avons constaté que cette fonction de restauration n'était pas implémentée dans la bibliothèque *opencv_dart*. Cette limitation technique bloquante nous a obligés à innover et à pivoter vers une solution par réseau de neurones avec le modèle génératif LaMa (Large Mask Inpainting).

La seconde étape a donc été, étonnamment, l'une des plus complexes de la gestion de l'IA. Faire tourner un réseau de neurones génératif sur un appareil mobile est un défi technique en termes de consommation de mémoire et de puissance de calcul. La solution d'optimisation que nous avons implémentée a été d'isoler uniquement la zone de travail. Au lieu de demander à l'IA de traiter l'image entière, le code calcule une zone de recadrage (représentant 2,5 fois la taille de l'autocollant pour donner du contexte spatial au modèle), redimensionne ce petit patch au format requis de 512x512 pixels, puis exécute l'inférence.

Cependant, la restitution de ce patch généré par l'IA a mis en évidence de sérieux problèmes de réalisme visuel. Le modèle génératif avait tendance à produire des réparations trop « lisses », présentant de légères bavures chromatiques ou un rendu géométrique incohérent, créant une démarcation artificielle flagrante avec le grain photographique naturel du reste du mur. Pour corriger cela de manière procédurale et sans intervention humaine, nous avons développé une série de post-traitements spécifiques. Avant l'inpainting, le masque de l'autocollant est informatiquement rempli avec la couleur moyenne environnante pour guider et faciliter le travail de l'IA. Après l'inpainting, le patch reconstruit subit un redimensionnement de haute qualité par interpolation Lanczos4, puis un filtre de netteté est appliqué pour récupérer les détails de la texture du mur. Enfin, nous injectons un bruit gaussien artificiel calculé pour reproduire fidèlement le grain du capteur de l'appareil photo. Ce n'est qu'après ce traitement rigoureux que le patch est fondu dans la photographie d'origine par une fusion adoucie des bords, rendant la disparition de l'autocollant totalement indiscernable à l'œil nu.

Mais même avec tout ça, ça n'est pas suffisant car y'a des images où ça marche excellemment bien mais d'autre pas cohérent.



Avant/Après de l'Inpainting

B. Traitement OpenCV

Une fois l'environnement préparé et l'autocollant effacé par l'Intelligence Artificielle LaMa, le cœur graphique de notre simulateur prend le relais. La bibliothèque OpenCV a été exploitée pour réaliser les opérations incrustation de l'équipement. L'objectif n'était pas d'effectuer un simple collage 2D, mais de générer procéduralement une réplique en trois dimensions de l'équipement sélectionné dans le catalogue, parfaitement intégrée à la géométrie de la pièce tout en étant réaliste.

1. Projection Perspective et Incrustation de Base

La première étape chronologique de ce traitement a consisté à calculer la déformation géométrique de l'équipement. À partir des quatre points détectés par l'IA et triés mathématiquement, nous avons calculé une matrice de transformation homographique via la fonction *getPerspectiveTransform()*. Cette matrice permet de déformer l'image 2D native de l'équipement (issue de notre catalogue) pour qu'elle épouse parfaitement l'angle de fuite et l'inclinaison du mur photographié. Les dimensions de l'équipement en pixels sont calculées dynamiquement grâce à une échelle de proportionnalité (ratio pixels/millimètres) déduite de la taille réelle connue de l'autocollant.



Première incrustation d'une climatisation avec la bonne taille respecter sur une image stock

2. Éclairage, Ombres Portées et Colorimétrie Adaptative

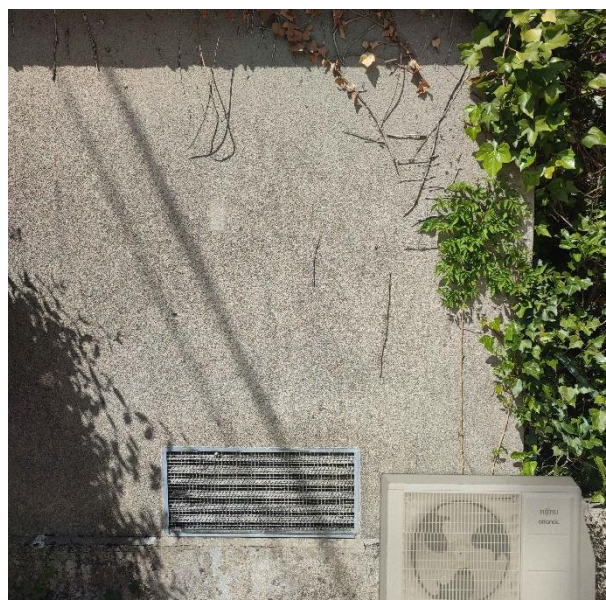
Pour fondre l'équipement dans son environnement, il était impératif de simuler l'éclairage de la pièce. Plutôt que d'appliquer une ombre générique, nous avons utilisé des filtres de Sobel sur la photographie du mur pour détecter la direction des gradients de luminosité. Cela permet au système de déduire d'où provient la source de lumière principale et de décaler l'ombre portée dans la direction opposée.

En parallèle, l'équipement subit une correction colorimétrique. L'algorithme prélève la couleur et la luminosité moyennes du mur se trouvant derrière l'appareil. Ces valeurs sont injectées dans le modèle colorimétrique HSV de la machine virtuelle pour qu'elle absorbe légèrement la teinte ambiante de la pièce et ajuste son exposition. De plus, le système prend en compte les ombres déjà existantes dans la pièce : si l'équipement est placé dans un endroit naturellement plus sombre ou dans un recoin ombragé, la luminosité de la coque s'adapte et s'assombrit localement pour s'intégrer parfaitement à cette zone sans créer de contraste artificiel. Enfin, un bruit gaussien est ajouté à la machine générée pour correspondre au bruit ISO naturel du capteur du smartphone.

3. Modélisation du Volume/Profondeur

Le principal défaut de la simple projection 2D était le manque de réalisme car les machines semblaient être de simples autocollant collés sur le mur. Pour pallier cela, nous avons développé un algorithme d'extrusion 3D dans lequel le code analyse le contour exact de l'image de la machine, calcule un point de fuite central (Qui est basé sur le centre de l'image pour respecter la distorsion de l'objectif) et génère des faces latérales pour créer l'effet de profondeur. La profondeur de cette extrusion dépend de la profondeur réelle en millimètres de l'équipement.

Cependant, une difficulté est apparue lors des tests sur les unités intérieures de climatisation. Contrairement aux unités extérieures qui sont des blocs très rectangulaires, les climatisation ont des façades arrondies et fluides. L'extrusion 3D créait des bords épais et durs qui cassaient totalement le réalisme. Pour résoudre ce problème, nous avons implémenté une double logique adaptative ou d'abord l'extrusion 3D est réduite si le ratio géométrique de l'image correspond à celui d'un équipement mural très large et ensuite nous appliquons une érosion matricielle (Une réduction de quelques pixels) sur le masque avant de générer les flancs 3D. Ainsi, le volume virtuel est construit en léger retrait par rapport à la face avant, cachant la jonction numérique sous la coque arrondie de l'appareil.



Incrustation d'une unité extérieure avec la profondeur/volume apparent en fonction de la position de celle-ci dans la photo

C. Création de l'Application et de l'Interface Utilisateur

Toute la complexité des algorithmes d'Intelligence Artificielle et des traitements matriciels d'OpenCV n'aurait aucune valeur commerciale si elle n'était pas encapsulée dans une application intuitive. L'objectif final étant de fournir un outil d'aide à la vente utilisable en clientèle par des artisans, l'interface devait masquer cette lourdeur technique derrière une fluidité et une esthétique irréprochables. Le développement sous le Framework Flutter s'est ainsi articulé autour de l'optimisation des performances, de la création d'outils interactifs avancés et d'un design moderne.

1. Architecture et Gestion d'État

La contrainte majeure de l'intégration d'OpenCV dans une application mobile réside dans le temps de calcul. Le langage Dart fonctionnant nativement de manière synchrone sur un thread unique, l'exécution de dizaines d'opérations matricielles figeait initialement l'interface utilisateur. Pour garantir une expérience sans latence, nous avons mis en place une architecture basée sur des *Isolates*. Ce mécanisme permet de déléguer la génération 3D et le rendu des ombres à des processus en arrière-plan, utilisant des cœurs de processeur distincts.

En parallèle, l'interface utilisateur a été optimisée via l'utilisation de *ValueNotifier*. Au lieu de recharger l'écran complet à chaque fois que l'utilisateur déplace un équipement avec son doigt, seuls les composants graphiques concernés sont mis à jour. Nous avons également programmé une gestion stricte du multi-touch, différenciant le glissement d'un objet avec un doigt, du zoom et du déplacement de la caméra avec deux doigts, annulant silencieusement les conflits gestuels pour éviter tout comportement erratique.

2. Outil de Capture Gyroscopique

La précision des calculs de perspective reposant sur la qualité de la photographie initiale, il était nécessaire d'assister l'artisan lors de la capture. Nous avons donc exploité l'accéléromètre du smartphone pour implémenter un « niveau à bulle » numérique en temps réel directement sur l'écran de l'appareil photo. Cet outil indique visuellement à l'utilisateur l'inclinaison latérale et frontale de son appareil, l'encourageant à se positionner parfaitement de face car on rappelle qu'on ne prend les photos que de face. Une fois le téléphone droit, le viseur passe au vert, garantissant des conditions optimales pour le moteur de traitement d'image.

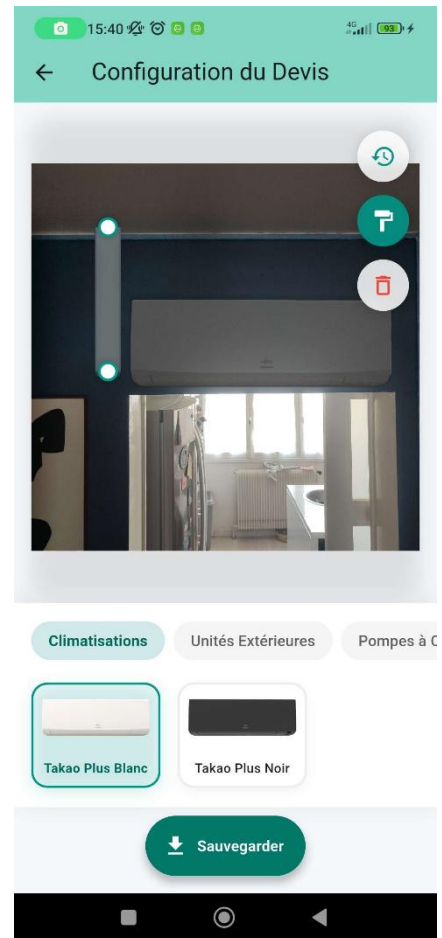


3. Incrustation des Goulottes

Pendant le développement de l'application, une fois l'équipement placé, on s'est dit que le simulateur devait permettre de tracer les goulottes de tuyauterie. L'implémentation graphique de cette fonctionnalité a nécessité de dessiner dynamiquement des segments orientés avec OpenCV. Pour éviter un rendu plat et irréaliste, les goulottes sont dessinées en superposant plusieurs lignes d'épaisseurs et de couleurs dégradées, créant ainsi un dégradé simulant un cylindre en plastique.

Lors du tracé, l'application enregistre l'angle du tout premier segment dessiné par l'utilisateur. Ensuite, le système contraint mathématiquement la ligne de manière à ce que chaque changement de direction se fasse exclusivement à 90 degrés par rapport à cet axe de référence.

Pour permettre la modification de ces goulottes après leur création, nous avons programmé un système nodal, on peut donc changer la taille de la goulotte en touchant les extrémité. Le déplacement d'un nœud central ajuste automatiquement les deux segments qui y sont rattachés pour préserver l'angle droit parfait, empêchant l'utilisateur de "casser" la structure. Enfin, pour assurer une précision millimétrique, une loupe de visée apparaît de manière dynamique au-dessus du doigt de l'utilisateur lorsqu'il interagit avec l'un de ces nœuds.



4. Interface Utilisateur

Pour maximiser l'espace de travail et l'immersion, nous avons opté pour une interface misant sur la légèreté visuelle. Au lieu d'utiliser des blocs de menus opaques et stricts qui masqueraient la photographie du mur, nous avons intégré des effets de transparence avec un léger flou d'arrière-plan. Ainsi, que ce soit lors des écrans de chargement ou pour les boutons d'outils flottants, l'utilisateur devine toujours la pièce en arrière-plan. Le catalogue des équipements, situé au bas de l'écran a été conçu avec des bords arrondis et des ombres douces, donnant l'impression que les éléments flottent au-dessus de l'image. L'ensemble de l'application respecte une charte graphique cohérente basée sur des teintes bleu-vert, conférant une identité à la fois technologique et rassurante.

Enfin, pour accentuer l'impact commercial lors des démonstrations chez les particuliers, nous avons développé des outils d'interaction pensés pour créer un effet de surprise. Le plus notable est l'intégration d'un curseur de comparaison interactif placé au-dessus de l'image. Ce dernier permet à l'artisan ou au client de balayer l'écran de gauche à droite avec son doigt pour comparer instantanément le mur nu d'origine avec le mur virtuel équipé de sa nouvelle climatisation et de ses goulottes. D'autres raccourcis ont été ajoutés de manière discrète, comme des boutons permettant de réinitialiser instantanément la position d'un équipement ou de supprimer une goulotte en un clic, garantissant ainsi que l'artisan puisse ajuster sa simulation en temps réel sans jamais perdre le fil de son argumentaire de vente.

V. Perspective d'Amélioration

Bien que l'application développée réponde à une grande partie du cahier des charges initial en offrant une simulation d'incrustation fluide, autonome et plutôt réaliste, ce simulateur pose les bases d'un outil qui peut encore évoluer. Plusieurs axes d'amélioration, tant sur le plan algorithmique qu'administrative, peuvent être envisagés pour enrichir l'expérience utilisateur.

Sur le plan de la vision par ordinateur et de l'Intelligence Artificielle, une évolution technique majeure consisterait à s'affranchir totalement de l'autocollant de référence. Cela supprimerait l'étape de préparation physique sur le terrain, fluidifiant encore davantage le flux de travail de l'artisan. Concernant le réalisme du rendu graphique, la gestion du premier plan reste un défi technique à explorer. Actuellement, l'équipement virtuel est invariablement superposé au-dessus de l'image. L'ajout d'un outil interactif de gomme permettrait à l'utilisateur d'effacer numériquement certaines parties de la machine générée pour donner l'illusion qu'elle se trouve derrière un objet réel situé au premier plan (comme un meuble, un rideau ou une plante). On ajoute à ça le fait d'avoir un meilleur rendu notamment dans des images où il y a un contre-jour. Et pour les goulottes, avoir la possibilité de tracer en une fois une goulotte qui peut être sous plusieurs forme au lieu de juste une ligne serait un très bon plus.

Enfin, sur le plan administratif, ce simulateur visuel pourrait être couplé au système de gestion de l'entreprise. L'intégration d'un module de génération de documents permettrait de compiler automatiquement les photographies de comparaison « Avant/Après », les caractéristiques techniques de la machine sélectionnée et le schéma des goulottes au sein d'un devis PDF complet, prêt à être signé par le client à l'issue de la démonstration.

VI. Conclusion

En conclusion, ce stage de fin d'études au sein de l'entreprise Chauffage Der Energies a été une très bonne opportunité pour mettre en pratique et approfondir mes compétences en intelligence artificielle, en traitement d'image et en développement logiciel dans un contexte professionnel et commercial concret. Il m'a donné l'occasion de concevoir de zéro une application mobile innovante, répondant à un besoin métier pour aider les clients à se projeter lors des devis.

Tout au long de ce projet, j'ai pu appliquer mes connaissances théoriques en apprentissage automatique et en traitement d'image, tout en découvrant et en maîtrisant de nouveaux outils technologiques comme le Framework Flutter pour le développement mobile, Roboflow pour annoter un dataset ou encore l'intégration de modèles TFLite embarqués. J'ai également appris à concevoir des architectures logicielles complexes et asynchrones pour contourner les limites matérielles des smartphones, à optimiser les performances d'un moteur de rendu mathématique avec OpenCV, et à travailler avec rigueur pour livrer un outil stable, photoréaliste et parfaitement adapté aux conditions du terrain.

La création de ce simulateur a permis d'offrir une expérience utilisateur fluide, rapide et visuellement très aboutie. J'ai particulièrement apprécié le défi que représentait la synergie entre les différentes technologies que ce soit la détection et l'effacement de l'autocollant par les réseaux de neurones, la génération procédurale des volumes et des ombres, la création de l'interface, ou encore le développement du système de routage interactif pour les goulottes.

Enfin, au-delà des aspects purement techniques, ce stage a été une expérience humaine très enrichissante. L'accueil de l'équipe et la compréhension des enjeux commerciaux ont donné du sens à

mon travail, en me montrant l'impact direct qu'une application bien pensée peut avoir sur la stratégie de vente d'une entreprise. Ce projet clôturant mon Master m'a définitivement conforté dans mon choix de poursuivre ma carrière dans le domaine de l'intelligence artificielle et de l'ingénierie logicielle en gardant toujours une attention particulière portée à l'expérience utilisateur et à l'utilité concrète des outils que nous concevons.